

MONGODB FUNDAMENTALS

1. What is MongoDB?

MongoDB is a NoSQL document database.

Instead of storing data primarily in rows and columns, MongoDB stores data in documents.

MongoDB documents use a JSON-like structure.

2. Database and Collection

A MongoDB database can contain multiple collections.

For example:

Database:
college

Collections:
users
courses
students

3. Document

A document represents a single record.

Example:

```
{  
  "name": "Rinki",  
  "age": 24,  
  "course": "Web Development"  
}
```

MongoDB stores documents in BSON format internally.

4. Collection

A collection contains multiple documents.

Example:

users collection:

```
{  
  "name": "Rinki",  
  "age": 24  
}
```

```
{
  "name": "Kuchupuchu",
  "age": 22
}
```

```
{
  "name": "Neha",
  "age": 24
}
```

5. Insert

A document can be inserted using insertOne().

Example:

```
db.users.insertOne({
  name: "Rinki",
  age: 24
});
```

Multiple documents can be inserted using insertMany().

6. Find

The find() method is used to retrieve documents.

Example:

```
db.users.find();
```

A filter can be provided.

Example:

```
db.users.find({
  age: 24
});
```

7. Update

The updateOne() method can update a document.

Example:

```
db.users.updateOne(
  { name: "Rinki" },
  { $set: { age: 25 } }
);
```

8. Delete

The `deleteOne()` method removes a document.

Example:

```
db.users.deleteOne({  
  name: "Rinki"  
});
```

9. MongoDB and Node.js

Node.js applications can communicate with MongoDB.

Mongoose is an ODM commonly used with MongoDB and Node.js.

Example:

```
const mongoose = require("mongoose");  
  
mongoose.connect(process.env.MONGO_URI);
```

10. Schema

A Mongoose schema defines the structure of documents used by a model.

Example:

```
const userSchema = new mongoose.Schema({  
  
  name: String,  
  
  age: Number,  
  
  email: String  
  
});
```

11. Model

A Mongoose model provides an interface for interacting with a MongoDB collection.

Example:

```
const User = mongoose.model(  
  "User",  
  userSchema  
);
```

12. Querying Data

A Mongoose model can be used to retrieve data.

Example:

```
const users = await User.find();
```

A specific user can be retrieved using:

```
const user = await User.findOne({  
  email: "rinki@example.com"  
});
```

13. Indexes

MongoDB indexes can improve query performance.

An index allows MongoDB to find data more efficiently without scanning every document.

14. Summary

MongoDB provides:

- Databases
- Collections
- Documents
- Queries
- Insert operations
- Update operations
- Delete operations
- Indexes

Mongoose can be used with Node.js applications to work with MongoDB using schemas and models.